

Reproducibility Study: Muon vs. ROOT Optimizer

Evaluating Robust Orthogonalization on Vision and Language Tasks

Dev Rawal

Ahmad Alhassan

April 27, 2026

Abstract

This report describes a reproducibility study that compares how well the Muon and ROOT optimizers work against a standard AdamW baseline. Muon is very efficient at computing because it uses Newton-Schulz iterations to orthogonalize matrix parameters. ROOT says that adding dimension-adaptive Newton iterations (AdaNewton) and soft-thresholding to Muon makes it better by cutting down on outlier gradient noise. We assess this assertion by developing both optimizers from the ground up and evaluating them in two separate domains: a Computer Vision task (ResNet-18 on CIFAR-10) and a Language Modeling task (NanoGPT). Our study monitors validation accuracy, perplexity, and computational overhead to assess whether ROOT’s dual robustness mechanisms yield the claimed enhancements in convergence and precision across modalities. On CIFAR-10, both orthogonalizers substantially outpace AdamW in early convergence ($>82\%$ vs. 74.52% at 10 epochs), and ROOT achieves the highest final accuracy (88.00%) and lowest validation loss (0.67723) after 100 epochs, suggesting that soft-thresholding provides a genuine regularization benefit at negligible additional cost over Muon ($\sim 1.5\%$ extra wall-clock time). On NanoGPT trained on BookCorpus, the improvements are present but modest: Muon attains the best validation perplexity (3.3987), followed closely by ROOT (3.4056) and AdamW (3.4135), with all three within a narrow band. We conclude that orthogonalized optimizers offer their strongest advantages in convolutional architectures with heavily structured 2D weight matrices, while gains in transformer-based language models at this scale remain marginal.

1 Introduction

Modern deep learning relies on element-wise adaptive optimizers like Adam and AdamW [1] [2]. While effective, these optimizers treat neural network weights as a collection of isolated scalars, ignoring the geometric and structural properties of weight matrices in hidden layers.

Recently, the Muon optimizer was introduced to orthogonalize matrix parameters using Newton-Schulz iterations, improving convergence and feature diversity [3]. But, Muon is sensitive to gradient noise and rigid regarding varying matrix dimensions. To address these vulnerabilities, the Robust Orthogonalized Optimizer (ROOT) was proposed, introducing soft-thresholding to filter noise and AdaNewton iterations to adapt to specific layer dimensions [4].

The objective of this project is to conduct a direct, side-by-side reproducibility study of Muon and ROOT to determine if the theoretical improvements hold up in practice across widely varying architectures.

2 Methodology

2.1 AdamW Baseline

AdamW serves as our baseline control. It provides parameter-wise adaptivity by scaling updates based on the moving average of squared gradients. It is highly robust to hyperparameter

variance, making it an ideal standard for measuring the "default" optimization trajectory of our architectures. For all experiments, 1D parameters (biases, normalization layers) are optimized using AdamW, as orthogonalization is strictly a 2D+ matrix operation.

2.2 Muon Optimizer

Muon (Momentum Orthogonalized by Newton-Schulz) operates strictly on 2D hidden layer parameters (Conv2D, Linear). Instead of applying raw momentum directly, Muon normalizes the momentum matrix and applies a 5-step Newton-Schulz iteration to approximate Singular Value Decomposition (SVD). This forces the update matrix to become orthogonal, preventing "lazy" dominant dimensions and encouraging the network to utilize its full parameter capacity evenly.

2.3 ROOT Optimizer

ROOT builds upon the Muon framework by integrating two specific robustness mechanisms:

1. **Soft-Thresholding (Proximal Optimization):** Before orthogonalization, ROOT applies a thresholding function to the momentum buffer, clamping extreme outliers and zeroing out microscopic gradient noise. This prevents noisy batches from destabilizing the Newton-Schulz iterations.
2. **AdaNewton Iterations:** Instead of using fixed polynomial coefficients for the Newton-Schulz steps, ROOT dynamically recalculates the optimal coefficients based on the aspect ratio of the target matrix, preserving orthogonalization precision across highly rectangular layers.

3 Experimental Setup

3.1 Computer Vision (ResNet-18)

To evaluate the optimizers on image data, we utilized the CIFAR-10 dataset with standard data augmentation techniques (Random Crop, Random Horizontal Flip) and normalization [5]. We trained a standard ResNet-18 architecture from scratch, modifying the final fully connected layer to output 10 classes [6]. All 2D parameters were routed to Muon/ROOT, while 1D parameters were routed to AdamW. We conducted a 10-epoch pilot run to verify early dynamics and a full 100-epoch run (with a Cosine Annealing scheduler) to evaluate final precision.

3.2 Language Modeling (NanoGPT)

We train a character-level NanoGPT [7]. The architecture has 4 layers, 4 attention heads, embedding width 256, block size 128, and dropout 0.1. We report two text sources: the standard *Tiny Shakespeare* split (90%/10% train/validation) and a streamed *BookCorpus* subsample [8] [9]. Each run uses 50,000 training steps, batch size 64, and validation every 250 steps (50 evaluation batches). One step consumes a batch of shape 64×128 , i.e. 8,192 token positions per step, for a total of about 4.1×10^8 token positions per run (sampled with replacement; character vocabulary). *Muon* and *ROOT* update only *hidden* 2D weight matrices; *AdamW* optimizes token embeddings, the language-modeling head, and all biases, layer norms, and other non-matrix parameters. The pure-AdamW baseline uses a single AdamW on all parameters. The learning-rate schedule is cosine decay with 1,000 warm-up steps and minimum LR scale 0.1. Results are averaged over seeds 42, 123, and 999. Orthogonalizers use the tuned Newton-Schulz and related settings from the best-practices implementation; ROOT additionally uses its adaptive inner iterations and soft-thresholding. Throughout, "perplexity" means exp of base-*e* validation cross-entropy.

4 Results and Reproducibility

4.1 Computer Vision Results

Early Convergence Dynamics: Our initial 10-epoch pilot runs demonstrated that orthogonalization provides an immediate and substantial acceleration in early-stage learning. By the end of 10 epochs, the standard AdamW baseline reached a validation accuracy of only 74.52%. On the other hand, both Muon and ROOT were past the 82% mark within the same timeframe, with Muon slightly better than ROOT at 82.50% compared to 82.01%. This confirms that forcing weight matrices to remain orthogonal dramatically improves the efficiency of early feature extraction in Convolutional Neural Networks.

Final Precision and Generalization: Scaling the training to 100 epochs showed the long-term stabilization benefits of ROOT. While AdamW converged to a best validation accuracy of 84.28%, both Muon and ROOT outperformed it. ROOT achieved the highest overall best validation accuracy at 88.00%, beating Muon’s 87.83%.

The validation loss curves in Figure 1 show ROOT’s robustness. Around epoch 30, all models began to overfit: validation loss rose steadily while training loss kept falling. ROOT’s final validation loss was 0.67723, lower than Muon’s 0.77666 and AdamW’s 0.86992. This gap suggests that ROOT’s soft-thresholding filters gradient noise, preventing the model from memorizing training artifacts and preserving better generalization.

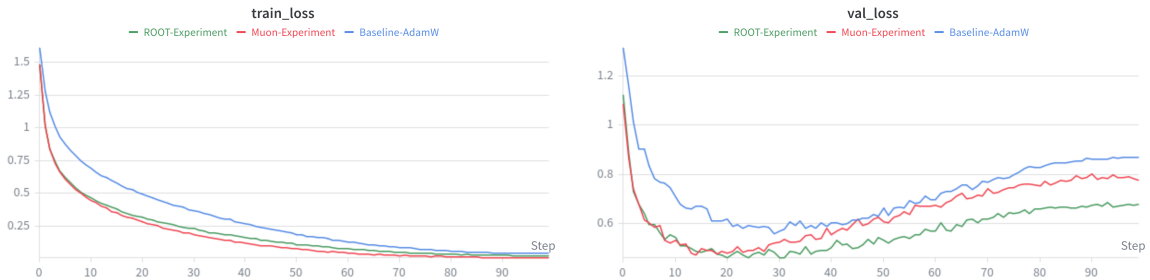


Figure 1: Training Loss (left) and Validation Loss (right) over 100 epochs. Note the distinct separation in validation loss after step 30, where ROOT remains the most resilient to severe overfitting.

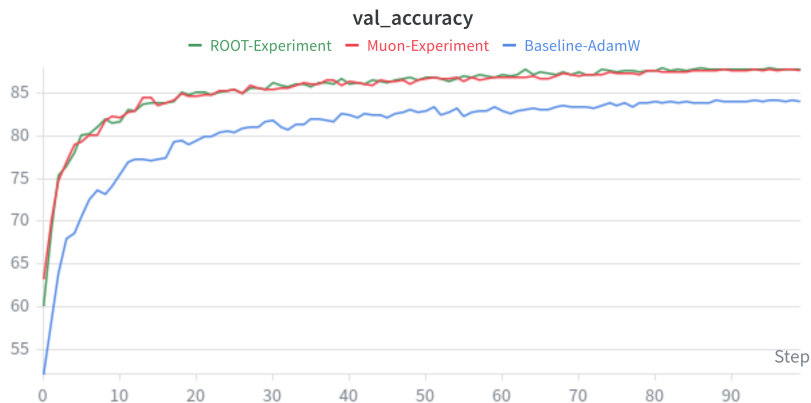


Figure 2: Validation accuracy over 100 epochs. Both Muon and ROOT establish an early lead and maintain a roughly 3-4% gap over AdamW throughout the training process.

Metric (100 Epochs)	AdamW	Muon	ROOT
Final Val Accuracy (%)	84.06	87.76	87.88
Best Val Accuracy (%)	84.28	87.83	88.00
Final Val Loss	0.86992	0.77666	0.67723
Total Runtime (s)	2258.02	4997.62	5073.20

Table 1: Final convergence and runtime metrics for ResNet-18 after 100 epochs.

Computational Efficiency: Muon and ROOT were approximately $2.2\times$ slower in total wall-clock time compared to AdamW (2258 seconds). The critical insight is the overhead between the two orthogonalizers: ROOT is only 1.5% slower than Muon (5073s vs 4997s). This minimal runtime gap shows that ROOT’s additional mechanisms, AdaNewton and soft-thresholding, add negligible overhead. Better final loss and accuracy come at essentially no extra cost beyond the initial Newton-Schulz orthogonalization.

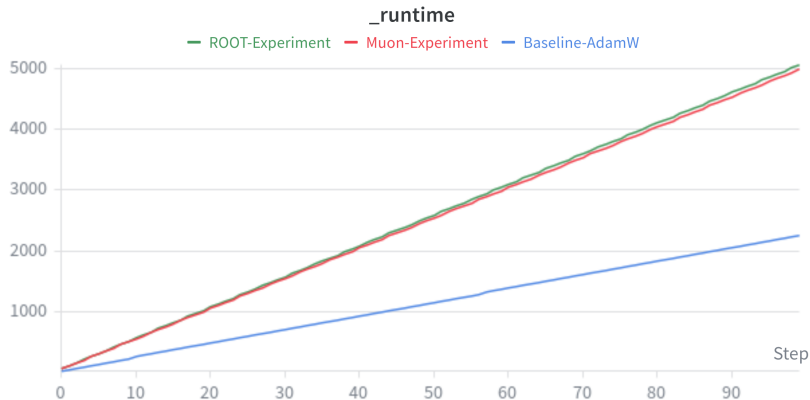


Figure 3: Total runtime in seconds over 100 epochs. The computational tax of ROOT over Muon is nearly indistinguishable visually.

4.2 Language Modeling Results

On **BookCorpus** (the larger, more reliable comparison for this model class), the best-practice aggregate tables show that **Muon** attains the lowest *best-checkpoint* validation perplexity, with **ROOT** and **AdamW** slightly but consistently higher. At the *final* step (50k), the ordering remains favorable to Muon, with all three within a small band on validation perplexity. This aligns with the project narrative: once Muon receives warm-up, cosine schedule, and correct routing, it is competitive with or better than AdamW on a corpus large enough to avoid instant saturation. On **Tiny Shakespeare**, best-checkpoint perplexities fall in a very tight range across optimizers. We therefore treat BookCorpus as the primary evidence and Tiny Shakespeare as supporting. **Stability and runtime.** AdamW exhibits the lowest per-step and total wall-clock time. Muon and ROOT incur additional cost from orthogonalization; in our logged runs, total time to 50k steps is substantially higher for Muon/ROOT than for AdamW on the same hardware class. Perplexity time series (e.g. confidence bands over seeds) are visually smoother for AdamW; orthogonalizers show comparable curve shapes but with higher time cost. Table 2 summarizes BookCorpus: final and best validation perplexity (multi-seed means from the project aggregate) and mean total runtime in seconds at step 50k over the three seeds. Figure 4 shows validation perplexity vs. step on BookCorpus with seed uncertainty.

Metric (NanoGPT, BookCorpus)	AdamW	Muon	ROOT
Final val perplexity	3.4516	3.4351	3.4410
Best val perplexity	3.4135	3.3987	3.4056
Total runtime (s)	1392.2	2138.1	2241.9

Table 2: BookCorpus, best-practices pipeline: final and best validation perplexity. The values are the means of training runs of 50,000 steps over seeds 42, 123, and 999.

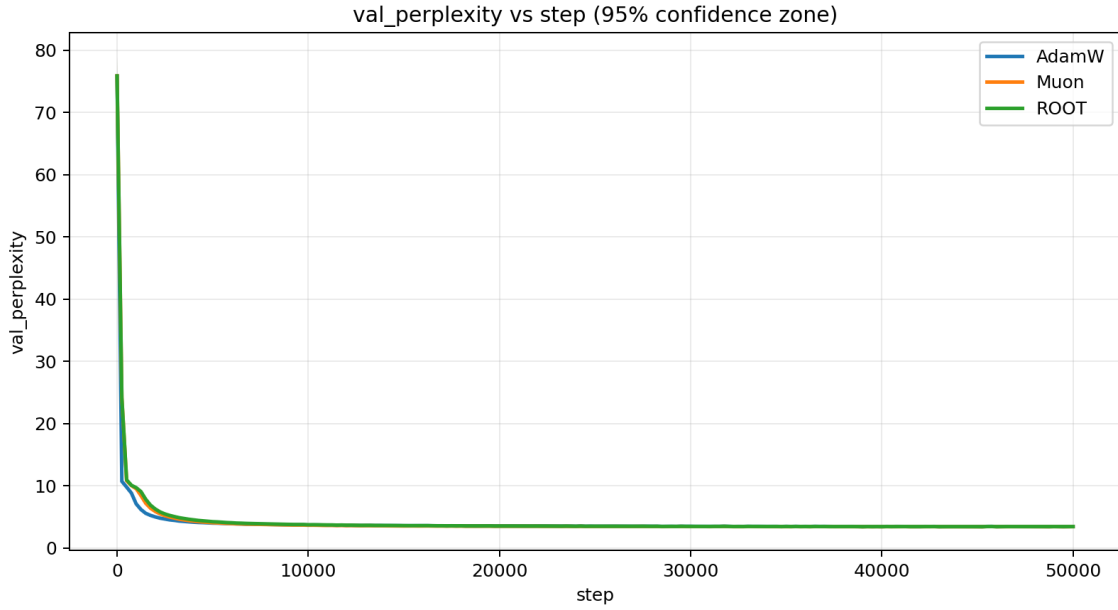


Figure 4: Validation perplexity vs. training step on BookCorpus.

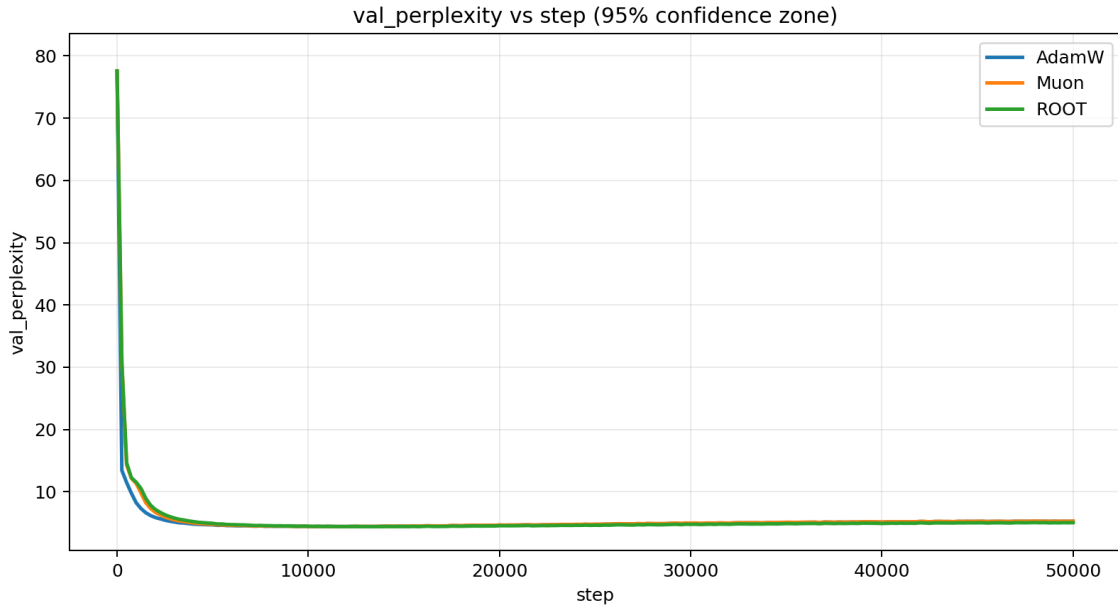


Figure 5: Same as Figure 4, for Tiny Shakespeare.

5 Conclusion

This reproducibility study set out to determine whether ROOT’s dual robustness mechanisms, soft-thresholding and AdaNewton iterations, deliver meaningful improvements over Muon and AdamW across vision and language modalities. Our findings reveal a nuanced, domain-dependent picture.

On the **Computer Vision** task (ResNet-18, CIFAR-10), the case for orthogonalized optimizers is clear and consistent. Both Muon and ROOT converge substantially faster in early training, reaching $>82\%$ validation accuracy within 10 epochs while AdamW reaches only 74.52%. Over the full 100-epoch run, ROOT achieves the best validation accuracy (88.00%) and the lowest validation loss (0.67723), suggesting that soft-thresholding provides a genuine regularization benefit by suppressing gradient noise that would otherwise destabilize Newton-Schulz iterations and encourage overfitting. Critically, ROOT incurs only $\sim 1.5\%$ additional wall-clock time over Muon (5073s vs. 4997s), making its improvements essentially free beyond the base cost of orthogonalization. Both orthogonalizers are approximately $2.2\times$ slower than AdamW, a cost that appears justified by the 3–4 percentage point accuracy gap they maintain throughout training.

On the **Language Modeling** task (NanoGPT, BookCorpus), the results are more modest. Muon achieves the best validation perplexity (3.3987), with ROOT (3.4056) and AdamW (3.4135) following closely. While the ordering is consistent with the orthogonalizer narrative, the absolute differences are small enough that practical significance is limited. Tiny Shakespeare results are similarly tight, reinforcing BookCorpus as the primary evidence. AdamW remains the fastest optimizer in this setting (1392s vs. 2138s for Muon and 2241s for ROOT), and its perplexity curves are visually smoother across seeds. This suggests that for character-level language models of this scale, the geometric structure exploited by orthogonalization is less decisive than in convolutional architectures, where spatial weight matrices benefit most from enforced orthogonality.

Taken together, our results **partially confirm** ROOT’s theoretical claims. The robustness mechanisms do provide measurable gains in generalization on vision tasks with negligible computational overhead, validating the design motivation. However, the improvements on language modeling are marginal, indicating that ROOT’s advantages are architecture- and modality-sensitive. We conclude that orthogonalized optimizers, particularly ROOT, are most impactful in settings with heavily structured 2D weight matrices (e.g., convolutional layers) and less so in transformer-based models where embedding and attention parameters constitute a larger share of the optimization landscape. Future work should explore ROOT’s behavior on larger-scale language models, where hidden projection matrices are more dominant, to determine whether the modality gap persists at scale.

6 Team Contributions

This reproducibility study was a collaborative effort, with experiments divided to cover both modalities:

- **Dev Rawal:** Led the **Computer Vision** domain. Responsible for setting up the ResNet-18 architecture, configuring the CIFAR-10 data pipeline, and executing both the 10-epoch pilot and 100-epoch full training regimes. Dev collected the vision-specific metrics, generated the associated accuracy and loss plots, and authored the CV experimental setup and results sections.
- **Ahmad Alhassan:** Led the **Language Modeling** domain. Responsible for configuring the character-level NanoGPT architecture, processing the Tiny Shakespeare and BookCorpus datasets, and managing the 50k-step multi-seed training runs. Ahmad collected

the language-specific metrics, generated the perplexity plots, and authored the NLP experimental setup and results sections.

Both team members collaborated equally on the core implementation of the Muon and ROOT optimizers from scratch, and co-authored the abstract, introduction, methodology, and conclusion of this report.

References

- [1] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [2] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.
- [3] K. Jordan, “Muon: An optimizer for hidden layers in neural networks,” *kellerjordan.github.io*, 2024. [Online]. Available: <https://kellerjordan.github.io/posts/muon/>
- [4] W. He *et al.*, “ROOT: Robust orthogonalized optimizer for neural network training,” *arXiv preprint arXiv:2511.20626*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.20626>
- [5] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Tech. Rep., 2009.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [7] A. Karpathy, “nanoGPT,” GitHub repository, 2022. [Online]. Available: <https://github.com/karpathy/nanoGPT>
- [8] A. Karpathy, “char-rnn,” GitHub repository, 2015. [Online]. Available: <https://github.com/karpathy/char-rnn>
- [9] Y. Zhu, R. Kiros, R. Zemel, R. Salakhutdinov, R. Urtasun, A. Torralba, and S. Fidler, “Aligning books and movies: Towards story-like visual explanations by watching movies and reading books,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pp. 19–27, 2015.